Chris Low Emma Li

Optimizing Cloud Virtual Machine CPU Reservations via Optimal Policy Trees

1 Motivation

Cloud computing providers such as Microsoft Azure operate large data centers that support millions of virtual machines (VMs). When a customer creates a VM, they request a certain amount of computing capacity, including virtual CPU cores and memory. However, the amount requested can be substantially larger than the amount that is ultimately used. Reserving the full requested capacity for every VM is therefore conservative, but can leave physical computing resources underutilized.

This motivates the problem of VM right-sizing. A cloud provider could use information available when a VM is created to estimate how much physical CPU capacity should be reserved for it. The decision involves a natural trade-off. Reserving more capacity provides greater protection against high CPU demand, while reserving less capacity allows the provider to use its physical infrastructure more efficiently. The challenge is to make this decision before the VM's future workload is known.

Our project asks whether historical workload information can be used to learn an interpretable reservation policy that reduces reserved CPU capacity while controlling the risk of under-provisioning. We will use the Microsoft Azure Public Dataset V2 [1], which contains workload information for approximately 2.7 million VMs observed over a 30-day period. Our main measure of CPU demand will be each VM's 95th-percentile (P95) CPU utilization, which captures periods of relatively high usage without relying entirely on a single maximum observation.

For each new VM, we will use information that would be available at the time of creation, including requested CPU and memory size, creation-time characteristics, and the historical CPU utilization of previously completed VMs from the same customer or deployment. We will consider four possible reservation levels: 25%, 50%, 75%, and 100% of requested CPU capacity.

The project will compare traditional prediction-based approaches with the prescriptive methods discussed in Lectures 10 and 11. In particular, we will study whether an Optimal Policy Tree (OPT) can directly learn an interpretable rule for choosing reservation levels and improve on simpler fixed or prediction-based policies.

2 Proposed Methods

Problem Formulation

For each VM i, we define:

- X_i: information available when the VM is created, including VM size, creation-time information, and historical utilization of previously completed VMs from the same subscription or deployment;
- $Y_i \in [0, 1]$: observed P95 CPU utilization as a fraction of requested capacity;
- $r_k \in \{0.25, 0.50, 0.75, 1.00\}$: the CPU reservation decision.

We define the modeled cost of assigning reservation level r_k to VM i as

$$C_{ik} = r_k + \beta \max(0, Y_i - r_k).$$

The first term penalizes reserved capacity, while the second penalizes CPU demand that exceeds the reservation. The parameter $\beta > 1$ determines how costly under-provisioning is relative to reserving additional capacity. Because this trade-off is not directly observed, we will evaluate several values of β.

Our goal is to learn a policy $\pi(X_i) \in \{0.25, 0.50, 0.75, 1.00\}$ that minimizes out-of-sample allocation cost.

Feature Construction

The reservation decision should only use information that would actually be known when a VM is created. We will therefore exclude post-deployment variables such as the VM's realized average, maximum, or P95 CPU utilization and its deletion time. The main features will include requested CPU and memory size, creation-time information, and historical workload statistics. These historical features will be constructed chronologically.

Prediction-Based Benchmarks

We will first model future P95 CPU utilization using standard regression approaches such as Ridge regression, CART, and Random Forest.

These methods provide a conventional predict-then-decide benchmark: $X_i \longrightarrow \widehat{Y}_i \longrightarrow r_i$.

After estimating future CPU demand, the prediction will be translated into a reservation decision. Because under-predicting CPU demand can be substantially more costly than over-predicting it, we will also consider prediction-based policies with a safety margin selected on validation data.

This provides a useful comparison with the prescriptive methods because minimizing prediction error does not necessarily minimize the eventual reservation cost.

Optimal Policy Trees

Our primary course method will be **Optimal Policy Trees** [2].

For every historical VM, the observed value Y_i allows us to calculate the modeled cost associated with each of the four reservation decisions. We can therefore construct four modeled costs, $C_{i,25}, C_{i,50}, C_{i,75}, C_{i,100}$, for each VM. OPT then directly learns a tree that maps VM characteristics to a reservation level by minimizing total modeled allocation cost:

$$\min_{\pi \in \Pi} \sum_{i=1}^{n} C\left(Y_i, \pi(X_i)\right),$$

where Π is a class of shallow decision trees.

We will select the tree depth and other complexity parameters using validation data, balancing out-of-sample performance with interpretability. The goal is to obtain a relatively simple policy that captures meaningful differences across VMs without creating an unnecessarily complex decision rule, so the resulting reservation rules remain easy to interpret.

Potential Challenges

A first challenge is data leakage. CPU utilization is observed after VM creation, so variables derived from future utilization cannot be used as predictors. We will construct all historical features using only information available before each VM is created and use chronological rather than random train/test splits.

A second challenge is that the true economic cost of under-provisioning is not provided in the dataset. We will therefore perform a sensitivity analysis over several values of β, rather than claim that one particular penalty is the correct operational value.

A third challenge is that we only observe CPU usage under the VM's original configuration. We do not know how its usage or performance would change under a different reservation level. We therefore assume that the observed P95 CPU utilization represents the VM's underlying demand when comparing reservation decisions. As a result, our analysis focuses on capacity planning rather than the actual performance effects of limiting CPU capacity.

Finally, many VMs in the trace are short-lived. Because P95 utilization for these VMs can be based on relatively few observations, we will report both per-VM results and results weighted by VM resource usage, and examine whether the conclusions change for longer-lived VMs.

3 Evaluation

We will divide the trace chronologically into training, validation, and test periods. Models will be fit using the earlier portion of the trace, tuning decisions will be made on the validation period, and all final comparisons will be reported on the latest VMs in the trace.

We will compare OPT against several benchmarks:

Method	**Purpose**
Always reserve 100%	Conservative no-right-sizing benchmark
Best fixed reservation	Tests whether personalization is useful at all
Historical-customer heuristic	Tests whether a simple rule based on past customer usage is sufficient
Ridge / Random Forest prediction	Standard prediction-based approach
Prediction with safety margin	Stronger predict-then-decide benchmark
Greedy policy tree	Tests whether global tree optimization improves on greedy splitting
Random Forest Cost Prediction	Predict the expected cost of each reservation level and choose the lowest-cost option; provides a strong benchmark for OPT
Perfect-information oracle	Gives a lower bound on the modeled cost achievable with future demand known

The main evaluation criterion will be the out-of-sample modeled cost:

$$\frac{1}{n}\sum_{i=1}^{n} C\left(Y_i, \pi(X_i)\right).$$

We will also report metrics that make the trade-off easier to interpret:

- average CPU capacity reserved;
- percentage of VMs whose P95 demand exceeds their reservation;
- average amount of CPU shortfall;
- unused reserved capacity; and
- percentage improvement relative to the best fixed reservation.

Because VM sizes and lifetimes differ, we will additionally report resource-weighted results so that very short-lived VMs do not dominate the evaluation simply because they are numerous.

Finally, we will examine the learned OPT itself: its depth, which features it uses, and which reservation levels appear in its leaves. This will let us evaluate whether OPT can retain much of the performance of more complex methods while producing a simple policy that a cloud operator could understand and implement.

References

[1] Microsoft, *Azure Public Dataset V2*, Azure Public Dataset.

[2] M. Amram, J. Dunn, and Y. Zhuo, *Optimal Policy Trees*, Machine Learning, 2022.

[3] D. Bertsimas and N. Kallus, *From Predictive to Prescriptive Analytics*, Management Science, 2020.

[4] D. Bertsimas, *From Predictive to Prescriptive Analytics*, Lecture 10, Massachusetts Institute of Technology, 2026.

[5] D. Bertsimas, *Optimal Policy Trees*, Lecture 11, Massachusetts Institute of Technology, 2026.

[6] D. Bertsimas and J. Dunn, *Machine Learning Under a Modern Optimization Lens*, Dynamic Ideas, 2019.